point2text: Local Text Generation Toward a Point in Semantic Space

point2text contributors - loopmath

Technical report - 29 October 2026

Abstract

We describe the first implemented point2text system: a local interface that turns a location on a fixed two-dimensional embedding map into a search objective for paragraph generation. A frozen sentence encoder maps an original technical corpus into 384-dimensional vectors; principal component analysis defines the displayed coordinates. A user-selected point retrieves nearby context for a frozen local language model. Independent candidate paragraphs are decoded, re-embedded by the ordinary encoder, and ranked by Euclidean distance in the fixed map. The current implementation is a generate-and-verify baseline, with explicit model identities, cancellation, and recorded results. We formalize its objective, distinguish map proximity from full semantic equivalence, report a small engineering validation, and identify the interfaces required for a future diffusion-guidance extension.

1 Task and system boundary

The intended interaction is spatial: a user explores a corpus, chooses a position among ideas, and asks for a paragraph nearby. A click supplies two numbers, not a full semantic specification. Many texts can have similar displayed coordinates; a location need not be attainable by any fluent text. The implemented system therefore searches for a good candidate and displays its measured remaining error. It does not claim an inverse of the projection or a guaranteed match.

The demo ships 100 original conceptual paragraphs across ten technical topics. These are authored examples rather than published abstracts, retrieved evidence, or a held-out benchmark. The local generator is an installed instruction model accessed through Ollama [3]. Generation uses nearby corpus text as context and changes no model weights. The current baseline uses autoregressive generation; the planned diffusion work is a separate component.

2 Canonical embeddings and a fixed map

Let C = {x_i} contain the corpus paragraphs and E be the frozen encoder. We use all-MiniLM-L6-v2 [1], following the sentence-embedding approach of Sentence-BERT [2]. The checkpoint is pinned to an immutable revision. Its ordinary tokenizer, attention-mask mean pooling, float32 CPU execution, and L2 normalization are shared by corpus and generated-text scoring. Inputs exceeding 256 tokens, including special tokens, are rejected before encoding rather than scored through a silently truncated prefix.

$$e_i = E(x_i) \in \mathbb{R}^{384}, \qquad \|e_i\|_2 = 1.$$

The encoder identity records its revision and preprocessing. Replacing the encoder or preprocessing creates a new map; numeric clicks are not transferable between models. The display is a view of this particular embedding space.

2.1 PCA geometry and the optimization objective

We center the corpus embeddings at their mean mu and compute a singular value decomposition. Let V contain the two leading orthonormal directions as columns. The implementation stores the actual fitted arrays, fixes an axis-sign convention, and uses ordinary unwhitened PCA. New text is projected without refitting the map:

$$\mu = \frac{1}{|C|}\sum_i e_i, \quad \pi(e) = V^T(e-\mu), \quad V^TV = I_2.$$

For a clicked target y*, every completed candidate is measured using the same Euclidean distance. The browser applies an isotropic viewport transform, so axis aspect ratio does not change the scientific metric. Expanding the viewport to show a result changes only the camera, not PCA coordinates.

$$d(x, y^*) = \|\pi(E(x)) - y^*\|_2, \qquad \hat{x} = \operatorname{argmin}_{x \in A} d(x, y^*).$$

Here A is the set of usable, completed, unique candidates actually produced within the selected attempt budget. The current system does not optimize an explicit full embedding target. Algebraically, unobserved directions remain free: e = mu + V y* + r for any residual orthogonal to V. Most such vectors need not correspond to text, and normalizing a reconstructed vector can change its projection. Residual priors are therefore an independent research question rather than a prerequisite for this baseline.

3 Neighbor-conditioned candidate search

For each target, the system finds the four nearest corpus paragraphs in map distance. Their topics, titles, and text form a common prompt context. The model is asked for one original, accurate 60-100 word technical paragraph, without headings, invented references, or copied sentences. This is a prompting preference, not an automatic guarantee of factuality or originality.

The user selects one to six generation attempts. Each attempt uses the same neighborhood context, a distinct seed, temperature 0.8, top-p 0.9, and a bounded output budget. Previous outputs are not fed into later prompts, and the baseline performs no target-conditioned logit update, revision loop, gradient step, or decoder training. Empty and duplicate text is skipped; an incomplete transport or length-limited output is reported as an error rather than accepted as a complete paragraph.

```
Algorithm 1: Generate and verify a paragraph
  context <- four nearest corpus paragraphs to target
  verified <- empty set
  for attempt in 1,...,B:
    stop if cancellation was requested
    text <- local_generator(context, seed + attempt - 1)
    skip empty or duplicate text
    point <- fixed_map(canonical_encoder(text))
    publish (text, point, distance(point, target))
  return closest published candidate and its measured error
```

The generation budget bounds attempts, not guaranteed accepted results. Candidate quality is supplied by the language model and prompt; distance selects among valid outputs. No distance threshold is currently interpreted as semantic success.

4 Implementation, identities, and observability

The Python package separates the corpus and encoder, numerical geometry, generator transport, search strategy, and web delivery. The map bundle exposes projection, nearest-context retrieval, and actual-text scoring. The generator returns decoded text; it does not decide whether a target was reached. Search publishes verified candidate values. The browser consumes text, coordinates, and scalar diagnostics without fitting projections or receiving model tensors.

A background worker keeps HTTP requests responsive while generation runs. One search is active per server process. The browser polls for completed candidates, can stop a search, and discards responses for abandoned targets. Cancellation interrupts the generator stream and preserves already verified candidates. Requests carry a map ID; results record map, encoder, and generator identities. Stale-map requests are rejected. The map fingerprint includes corpus, encoder configuration, metric, embeddings, and fitted arrays.

Each search stores a JSON record containing target, map ID, candidate text and coordinates, model tag and installed digest, seeds, strategy, and elapsed time. These are inspection artifacts, not a guarantee of bitwise replay across runtime versions. The wheel includes the corpus and browser assets. A direct Tailscale binding permits remote access to the same local application while Ollama remains on loopback; it does not modify existing Serve routes.

5 Initial engineering validation

We validated the first demo on an Apple M4 Max with 128 GiB unified memory and Python 3.12.12. The sentence encoder ran on CPU. An existing local gemma4:12b installation supplied generation; no generator training or parameter update occurred. The corpus's longest encoder input was 62 tokens. PC1 and PC2 retained 6.11% and 5.62% of embedding variance, respectively, emphasizing that the map omits most full-space variation.

Check	Observed result
Unit and API fixtures	30 passed for the initial baseline
Real browser search	3 completed paragraphs in 11.70 s
Candidate lengths	90, 91, and 90 words
Actual map distances	0.05324, 0.08512, 0.14634
Independent re-scoring	All 3 coordinates/distances matched
Cancellation	Stopped at 5.44 s; 1 verified result retained

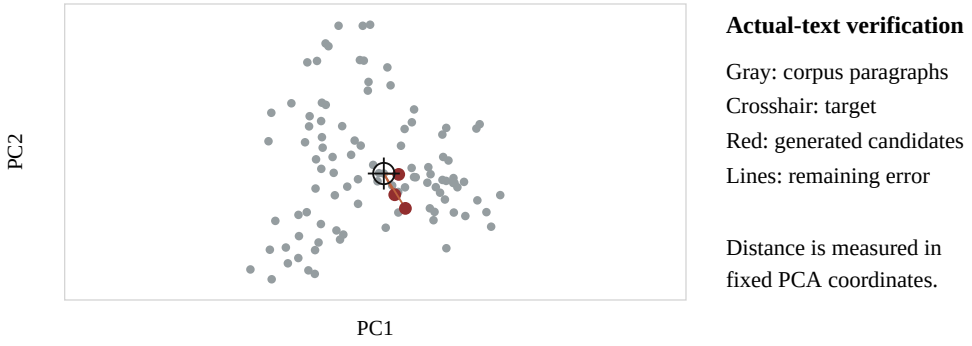


Figure 1. The fixed corpus map and three actual generated positions from the demonstration. Most embedding variance lies outside this two-dimensional view.

The target was the displayed position of the corpus's distribution-shift paragraph. Its neighborhood was available to the generator. The run demonstrates the complete interaction, not held-out reconstruction accuracy. Timing is one warm-model observation, not a latency distribution. The package also installed and scored text outside the source checkout, and CI checked fixture tests and wheel resources on Linux and macOS.

6 Limits and interpretation

The ranking objective is explicitly two-dimensional. Two paragraphs can be close in this map yet differ substantially in omitted dimensions, factual content, genre, or quality. A low map distance alone is insufficient evidence of semantic equivalence. The small authored corpus, neighboring context, and single demonstrated target do not establish generalization, unbiased quality, diversity, adversarial robustness, or reachability of arbitrary clicks. Sparse regions are exploratory targets; a point inside an observed hull is not thereby proven attainable.

The present search is limited by proposals from the language model. Increasing the attempt budget can improve the best observed distance, but does not guarantee progress. The backend rejects over-limit encoder inputs and unfinished outputs; it does not perform blind coherence review or verify technical assertions. Cold-start latency, peak memory, smaller Mac tiers, and a broad target panel remain to be measured. The server is a single local worker, without multi-user tenant isolation or distributed scheduling.

7 From this baseline to diffusion guidance

The next implementation is divided into orthogonal components: a native diffusion runtime and pre-commit sampler hook; a frozen encoder-input-gradient adapter and explicit tokenizer bridge; and a guidance/search layer that retains canonical final verification. DiffusionGemma [4] provides a concrete block-diffusion candidate. Native sampling, confidence-based commitment, and self-conditioning must all consume the intended guided proposal. An identity hook must reproduce unmodified execution under the same conditions.

The proposed intervention detaches clean-token logits z from the frozen generator, forms a differentiable token surrogate, and applies a small number of input-gradient updates through the encoder. A map loss favors the clicked point, while a KL term limits departure from the generator proposal:

$$L(z) = \|\pi(E_{\text{soft}}(z)) - y^{*}\|_2^2 + \lambda\,\mathrm{KL}(q_z \| q_0).$$

This is a local surrogate, not an exact derivative through decoded text. Different tokenizers, top-k truncation, and mixtures of token vectors introduce approximation error. Entropy-aware hard/soft guidance [5] is a relevant later comparison, not a proven solution to this map objective. Soft progress must be checked against actual decoded-text progress.

The effectiveness gate compares guided and unguided runs of the same diffusion model with equal prompt information, output constraints, hardware, and end-to-end time budgets. A working hook and finite gradients establish plumbing; useful guidance additionally requires improved actual-text error at acceptable coherence, diversity, and latency. Residual priors, acceleration, and alternate encoders follow as separate ablations. The current demonstration supplies the scoring boundary and baseline against which those claims can be tested.

References

[1] Sentence Transformers. all-MiniLM-L6-v2 model card. Pinned revision 1110a243, accessed 29 October 2026. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[2] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP, 2019. https://arxiv.org/abs/1908.10084

[3] Ollama. Generate a response: API documentation. Accessed 29 October 2026. https://docs.ollama.com/api/generate

[4] Google. DiffusionGemma model card. Accessed 29 October 2026. https://ai.google.dev/gemma/docs/diffusiongemma/model_card

[5] Tejaswi et al. Entropy Aware Reward Guidance for Diffusion Language Model Alignment. Preprint, v2, 2026. https://arxiv.org/abs/2602.05000